Encode-decode for truncations of loop spaces

The Univalent Foundations Program

hott-book:homotopy.tex:local-snapshot-20261011; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author's unlabelled lem environment at pinned-original-source.tex lines 2613-2632, the second statement of section 8.9 ("A general statement of the encode-decode method"), whose optional argument is [Encode-decode for Truncations of Loop Spaces]; the proof environment opens at line 2634, so the pairing is by adjacency.

Editorial note. Numbering: the lemma carries no label, so its number is part of its identity. The section begins at source line 2542 and contains exactly two theorem-class environments, the first being 'lem:encode-decode-loop' at line 2571; the retained counter values (chapter 8, section 9, theorem offset 1) therefore print the counted lemma as 8.9.2. The first-edition label-number dump carries no entry for either of the two labels of this section, so the number is a section-internal count rather than a dump-confirmed number, and is reported with that caveat.

Editorial note. The only labels used inside the retained range are the two item labels declared in the statement itself, so the package declares no external reference.

Editorial note. The retained preamble is the mechanical standalone adapter followed by the book's own macro library, reproduced verbatim from the same pinned source tree.

Editorial scope. One original result of the pinned first-edition chapter, counted once: the encode-decode statement for truncations of loop spaces (unlabelled lemma, original Lemma 8.9.2).

Allowed standard prerequisites (editorial).

- Encode-decode data for a pointed type. A fibration code:A->type with code(x) a k-type, a point c0:code(a0), a decoding map decode and the two round-trip identifications.. They are the hypotheses of the statement.
- Truncation recursion and induction. A map out of trunc k (id a0 x) may be defined by recursion when its target is a k-type, and a k-type indexed goal may be proved by induction on the truncation.. The encode map is defined by recursion and the second round-trip is proved by induction.
- k-type. A type is a k-type when all its path types are trivial above level k; in particular paths in a k-type form a k-type.. It is the reason the truncation induction of the proof is allowed, and the reason the recursion defining encode is legitimate.

Lemma 8.9.2 (Encode-decode for Truncations of Loop Spaces)**.** *Assume a pointed type* (A, a_0) *and a fibration* $\mathsf{code} : A \to \mathcal{U}$*, where for every* x*,* $\mathsf{code}(x)$ *is a* k*-type. Define*

$$\mathsf{encode} : \prod_{x:A} \|a_0 = x\|_k \to \mathsf{code}(x)$$

by truncation recursion (using the fact that $\mathsf{code}(x)$ *is a* k*-type), mapping* $\alpha : a_0 = x$ *to* $\mathsf{transport}^{\mathsf{code}}(\alpha, c_0)$*. Suppose:*

(i) $c_0 : \mathsf{code}(a_0)$*,*
(ii) $\mathsf{decode} : \prod_{(x:A)} \mathsf{code}(x) \to \|a_0 = x\|_k$*,*
(iii) $\mathsf{encode}_{a_0}(\mathsf{decode}_{a_0}(c)) = c$ *for all* $c : \mathsf{code}(a_0)$*, and*
(iv) $\mathsf{decode}(c_0) = |\mathsf{refl}|$*.*

Then $\|a_0 = a_0\|_k$ *is equivalent to* $\mathsf{code}(a_0)$*.*

Proof. That $\mathsf{decode} \circ \mathsf{encode}$ is identity is immediate by (iii). To prove $\mathsf{encode} \circ \mathsf{decode}$, we first do a truncation induction, by which it suffices to show

$$\prod_{(x:A)} \prod_{(p:a_0=x)} \mathsf{decode}_x(\mathsf{encode}_x(|p|_k)) = |p|_k.$$

The truncation induction is allowed because paths in a k-type are a k-type. To show this type, we do a path induction, and after reducing the encode, use assumption (iv). □